

HEP Agent Simulation Framework

Code Documentation

June 22, 2026

Contents

1	Introduction	4
1.1	Overview	4
1.1.1	Scientific Goal	4
1.1.2	Agent-Based Modeling	4
1.2	Architecture	5
1.2.1	Execution Control Flow	5
1.3	Installation	7
1.3.1	Requirements	7
1.3.2	Building the Project	7
1.4	Running the Simulation	7
1.4.1	Python UI (Recommended)	7
2	Developing New Modules	8
2.1	Overview	8
2.2	Path A: Independent Module Development	8
2.2.1	Step 1: Implement the Logic (Fortran)	9
2.2.2	Step 2: Register the Module ID	10
2.2.3	Step 3: Add the Dispatch Case	10
2.2.4	Step 4: Expose in the UI	10
2.2.5	Step 5: Rebuild & Test	10
2.3	Path B: Collaborative Workflow (Test Modules)	10
2.3.1	Step 1: Open the Test Module File	11
2.3.2	Step 2: Write Your Logic	11
2.3.3	Step 3: Rebuild & Test	12
2.3.4	Step 4: Iterate	12
2.3.5	Step 5: Hand Off to Daniel	12
2.4	Design Philosophy: Infrastructure vs. Science	12
2.5	Design Philosophy: Infrastructure vs. Science	12
2.5.1	1. Internal Infrastructure (Core Mechanics)	13
2.5.2	2. Interface Infrastructure (The Bridge)	13
2.5.3	3. Science (Simulation Modules)	13
2.6	Clustering and Spatial Analysis	14

2.6.1	K-Means Clustering	14
2.6.2	DBSCAN (Density-Based Spatial Clustering)	14
2.6.3	Convex Hull Fill	15
2.7	Visualization and Rendering	15
2.7.1	Rendering Architecture	15
2.8	Best Practices	15
3	Data Structures & Storage	16
3.1	Data Storage Logic	16
3.1.1	Design Choice: Hybrid Storage	16
3.1.2	The Agent Array	17
3.1.3	Spatial Indexing: The Grid	17
3.2	Global State & Metrics	18
3.2.1	Tick Accumulators (<code>t_tick_accumulators</code>)	18
3.2.2	Dynamic State (<code>t_dynamic_state</code>)	18
3.3	Module <code>mod_agent_world</code>	19
3.3.1	Agent Lookup: Hashmap	19
3.4	Compaction	19
3.5	Density Analysis & Clustering	20
3.5.1	Global Density Smoothing	20
3.5.2	Algorithms	20
3.5.3	Configuration	20
4	Fortran Only Version	21
4.1	Building the Executable	21
4.1.1	Prerequisites	21
4.1.2	Build Command	21
4.2	Running the Simulation	22
4.2.1	Execution	22
4.2.2	Command Line Arguments	22
4.3	Configuration	22
5	Recent Changes	23
6	Project State Summary - May 5, 2026	24
6.1	Objective	24
6.2	Completed Work	24
6.2.1	1. Dead Agent Archive & NetCDF Storage (April 21st)	24
6.2.2	2. New Module Pattern & World Data Access (April 23rd)	25
6.2.3	3. DBSCAN, Convex Hull & Performance Metrics (April 28th)	25

6.2.4	4. Unified Density Smoothing & Global Metrics (April 29th)	25
6.2.5	5. Birth-Death Refactoring & Logic Consolidation (May 4th)	26
6.2.6	6. Architectural Refinement & Critical Fixes (April - May)	26
6.3	TODO later	27
6.3.1	7. Cluster Population Control & Multi-Population Graphing (May 13th)	27
6.3.2	8. Performance Metrics HUD & Population-Specific Mating (May 26th)	28
6.3.3	9. Critical Bugfix: Hashmap Population Corruption (May 26th)	29
6.3.4	10. Efficient Cluster-Restricted Density Updates (May 26th)	29
6.3.5	11. Responsive Rolling Average for Performance HUD (May 26th)	31
6.3.6	12. HEP Data Chunked Loading (May 26th)	31
6.3.7	13. Clustering Fix: “Clusters Way Too Small” (May 26th)	32
6.3.8	14. Bugfix: Out-of-Bounds Array Access in Cluster Accumulators (May 26th)	33
6.4	Tripwires & Lessons Learned	33

Chapter 1

Introduction

1.1 Overview

The HEP Agent Simulation Framework (**ABM-Hescor**) is a high-performance simulation environment designed to model agent dynamics over large-scale geospatial grids (Human Evolution Potential - HEP data). The core simulation logic is written in modern **Fortran 2003/2008** for maximum performance, while the user interface and high-level control are implemented in **Python** (via PyQt5).

1.1.1 Scientific Goal

The present program is intended to simulate various aspects of the lives of prehistoric humans—such as Neanderthals and other human species. These aspects include movement, life cycle (reproduction and death), cultural development, and genetic inheritance. The goal is to compare the results of the simulation with archaeological data and then use the simulation data to fill gaps in the fossil record. In the past, a predecessor of this simulation already demonstrated, using climate data, how the settlement of Europe by modern humans might have taken place.

1.1.2 Agent-Based Modeling

The simulation is an agent-based model (ABM). This means that individual acting entities within the simulation are treated separately, and the agents' decisions are not aggregated but made individually for each agent. For example, rather than solving a partial differential equation (PDE) for population density, the model simulates millions of individual agents moving, reproducing, and interacting.

Note for Researchers: If you are primarily interested in the scientific modeling aspects (equations, biological rules, environmental interactions), the chapter "**Developing New Modules**" is the most important part of this document for you.

1.2 Architecture

The system follows a hybrid architecture (Fortran + Python):

- **Fortran Backend:** Handles all heavy lifting including:
 - Memory management for millions of agents (using `mod_agent_world`).
 - Spatial indexing (Grid) and Agent-ID mapping (Hashmap).
 - Simulation modules (Movement, Births, Deaths, Resources).
 - High-speed I/O for HEP NetCDF files.
- **Python Frontend:** Provides a user-friendly interface for:
 - configuring simulations.
 - Real-time visualization of agents and data on a 3D/2D globe.
 - Controlling the simulation loop (Start, Stop, Step).
 - Debugging and verification.

Interaction between the two layers is handled via `f2py` generated wrappers, exposing Fortran subroutines directly to Python as a compiled extension module (`mod_python_interface`).

1.2.1 Execution Control Flow

The lifecycle of the simulation (initialization and stepping) is managed entirely by the Python layer invoking subroutines in the Fortran backend.

Initialization

The Fortran backend is initialized in a specific sequence of step calls to set up configuration pathways, allocate memory blocks, generate grids, and spawn the starting agent populations.

- **Configuration Setup:** Prepares configuration paths and custom HEP file listings.

- **Module Registration:** Sets the array of active simulation module IDs.
- **Sequence Steps:**
 1. `init_sim_step_1`: Allocates global world parameters.
 2. `init_sim_step_2_part_1`: Parses config namelist parameters.
 3. `init_sim_step_2_part_2_arrays_only`: Prepares memory arrays for the geospatial grid.
 4. `init_sim_step_2_part_2_chunk(start, end)`: Populates grid values in chunks to keep the Python runtime responsive.
 5. `init_sim_step_2_part_3`: Reads environmental NetCDF input datasets.
 6. `set_spawn_configuration` (optional): Allocates spatial spawn coordinates if custom locations are selected.
 7. `init_sim_step_3`: Generates the agent population structure.
 8. `init_sim_step_apply_age_dist` (optional): Overrides default age structure.
 9. `init_sim_step_4`: Performs a final validation check on variables and allocations.

In interactive mode, this flow is executed in `prepare_simulation()` inside `python/application.py`. In headless runs, it is executed inside `run_simulation_procedure` in `python/headless_simulation.py`.

Simulation Step (Tick)

Stepping the simulation forward executes all active biological, movement, and demographic subroutines for a single time unit (tick).

- **Execution Call:** Exposes the Fortran step method as:

```
mod_python_interface.step_simulation(tick_index)
```

- **Event Loop:** In interactive mode, a Qt timer triggers `update_simulation()` inside `python/simulation.py`, which updates the tick counter and calls the Fortran stepping routine. In headless mode, this step runs sequentially inside a CPU-bound loop in `python/headless_simulation.py`.

1.3 Installation

The program requires Python, Fortran (gfortran), and pip.

1.3.1 Requirements

```
1 sudo apt install gfortran
2 sudo apt install python3 python3-venv python3-pip
```

Python Packages

Visualisation requires:

```
1 pip install pandas matplotlib pillow cartopy numpy imageio
```

1.3.2 Building the Project

To compile the Fortran extension and link it to Python:

1. Navigate to the project root.
2. Run the build script:

```
1      ./build_extension.sh
2
```

3. Verify that `mod_python_interface*.so` exists in the root directory.

1.4 Running the Simulation

1.4.1 Python UI (Recommended)

The preferred way to run the simulation, especially for development and debugging, is via the Python user interface:

```
1 python3 python/application.py
```

This opens a GUI that allows you to:

- Load config files and HEP maps.
- Start/Stop/Restart the simulation.
- Visualize agent positions in real-time.
- Select and verify active simulation modules.

Chapter 2

Developing New Modules

2.1 Overview

A “Module” in this framework is a self-contained behavior that can be applied to agents or to the grid (e.g., Movement, Aging, Mating). Modules are written in Fortran, registered in the interface layer, and toggled on/off from the Python UI.

There are **two ways** to develop new simulation logic, depending on your experience level and workflow preference:

Path A — Independent Development

You create a full module from scratch: write the Fortran subroutine, register it, wire it into the dispatch loop, and add it to the UI. Best for users comfortable with the full codebase.

Path B — Collaborative (Test Modules)

You write your logic directly in the pre-wired `test_modules.f95` file, test it, and send the file to Daniel. Daniel then turns your prototype into a proper, permanent module. Best for users who want to focus on the science without touching multiple files.

2.2 Path A: Independent Module Development

Use this path when you are confident editing multiple Fortran and Python files. Figure 2.1 shows the full workflow.

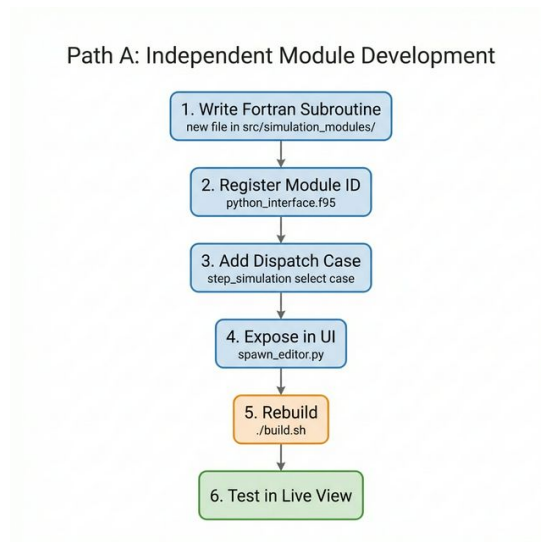


Figure 2.1: Path A: Creating a standalone module requires changes in four files.

2.2.1 Step 1: Implement the Logic (Fortran)

Create a new subroutine. There are two patterns depending on what your module operates on:

Agent-Centric Module

Runs once per living agent. The subroutine receives a pointer to the current agent.

```

1  subroutine my_agent_behavior(agent_ptr)
2      use mod_agent_world
3      implicit none
4      type(Agent), pointer, intent(inout) :: agent_ptr
5
6      ! Example: age-based death
7      if (agent_ptr%age > 3000) then
8          call agent_ptr%agent_dies(reason=6)
9      end if
10 end subroutine my_agent_behavior

```

Grid-Centric Module

Runs once per tick over the spatial grid. Receives the full world container.

```

1  subroutine my_grid_behavior(w)
2      use mod_agent_world
3      implicit none
4      class(world_container), target, intent(inout) :: w
5
6      integer :: gx, gy
7      do gy = 1, w%config%dlat_hep

```

```

8         do gx = 1, w%config%dlon_hep
9             ! operate on w%grid%cell(gx, gy)
10        end do
11    end do
12 end subroutine my_grid_behavior

```

2.2.2 Step 2: Register the Module ID

In `src/interfaces/python_interface.f95`, define a new integer constant. Pick the next available number:

```

1 integer, parameter :: MODULE_MY_BEHAVIOR = 17

```

2.2.3 Step 3: Add the Dispatch Case

In the `step_simulation` subroutine, add a case to the `select case` block:

```

1 case (MODULE_MY_BEHAVIOR)
2     ! For agent-centric:
3     call apply_module_to_agents(my_agent_behavior, t)
4     ! For grid-centric:
5     ! call my_grid_behavior(world)

```

2.2.4 Step 4: Expose in the UI

In `python/spawn_editor.py`, add your module to the `available_modules` dictionary (ID must match the Fortran constant):

```

1 self.available_modules = {
2     # ... existing modules ...
3     "My New Behavior": 17,
4 }

```

2.2.5 Step 5: Rebuild & Test

```

1 ./build.sh
2 python3 python/application.py

```

Add your module from the Spawn Editor and run the simulation to verify.

2.3 Path B: Collaborative Workflow (Test Modules)

Use this path when you want to **focus on writing your logic** without needing to touch the interface, build scripts, or registration code. Everything is already wired up for you.

Figure 2.2 shows the collaborative workflow.

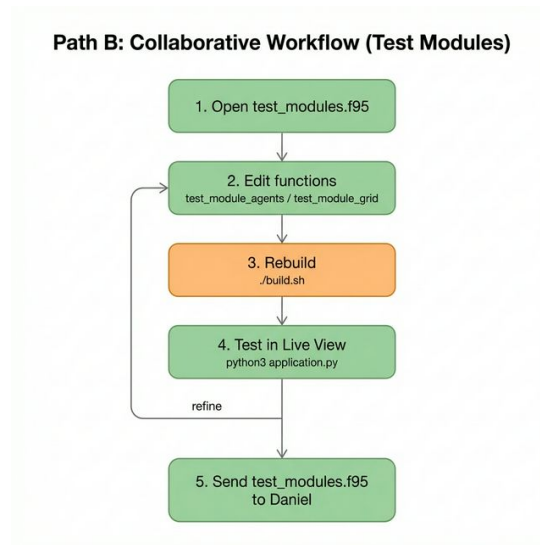


Figure 2.2: Path B: Edit the test module, rebuild, test, and iterate until ready.

2.3.1 Step 1: Open the Test Module File

Open `src/simulation_modules/mod_test_modules.f95`. You will find two subroutines:

`test_module_agents` Runs your code on **every living agent**, once per tick.

`test_module_grid` Runs your code **once per tick** on the entire grid.

Choose the one that fits your use case (or use both).

2.3.2 Step 2: Write Your Logic

Each subroutine has a clearly marked section:

```

1 ! =====
2 ! DECLARE YOUR VARIABLES HERE
3 ! =====
4
5 ! =====
6 ! YOUR CODE HERE
7 ! =====

```

Important:

- Declare all variables you need at the top of the subroutine (after `implicit none`).

- You do **not** need to modify any config files.
- Use `print *`, `"..."` followed by `call flush(6)` for debug output.
- The file contains detailed comments listing all available agent fields, grid accessors, and example code.

2.3.3 Step 3: Rebuild & Test

```
1 ./build.sh
2 python3 python/application.py
```

In the Spawn Editor, add “**Test Module (Agents)**” or “**Test Module (Grid)**” to the active module list, then run the simulation. Check the terminal for your `print` output.

2.3.4 Step 4: Iterate

Repeat steps 2–3 until your results are correct. When you are satisfied, move on to step 5.

2.3.5 Step 5: Hand Off to Daniel

Send your modified `mod_test_modules.f95` file to Daniel. He will:

1. Review and optimize your code.
2. Extract it into a proper, standalone module with its own file and config parameters.
3. Merge it into the main codebase so it becomes a permanent, selectable module.

2.4 Design Philosophy: Infrastructure vs. Science

2.5 Design Philosophy: Infrastructure vs. Science

A core architectural principle of the HEP Agent Simulation Framework is the separation of **infrastructure** (background mechanics) from **science** (model logic and equations).

However, this boundary is fluid. In practice, functions generally fall into three distinct categories:

2.5.1 1. Internal Infrastructure (Core Mechanics)

These functions manage memory arrays, update spatial grid configurations, or handle raw processing. They belong directly to the core data structures (e.g., `mod_agent_world` or `mod_grid_id`) and contain **no scientific equations**. They exist solely to maintain the integrity of the computer state.

Examples of Internal Infrastructure:

- `setup_grid`: Allocates the memory arrays for the simulation.
- `place_agent_in_grid` / `remove_agent_from_cell`: Updates spatial indices when an agent moves.
- `get_male_from_cell`: A technical helper procedure to retrieve data from memory.

2.5.2 2. Interface Infrastructure (The Bridge)

These are foundational functions that bridge the gap between memory management and scientific rules. While they belong to the structural code (like `mod_agent_world`), they are typically **called by science functions** to enact a biological event formally within the system.

Examples of Interface Infrastructure:

- `generate_agent_born`: Validates parents, initializes genetic tracking, and spawns the structural **Agent** entity into the world memory array.
- `agent_dies`: Formally flags the agent as dead, increments mortality tracking counters, and performs structural cleanup.

2.5.3 3. Science (Simulation Modules)

Functions that determine *why*, *when*, or *how* an agent behaves based on biological, ecological, or physical rules. These belong in the **Simulation Modules** layer (the "science" layer). These routines contain the actual mathematical mortality/birth equations and then call the Interface Infrastructure to execute the structural changes.

Examples of Science Procedures:

- **reviewed_agent_motion**: Uses the Langevin equation, random diffusion, and spatial gradients to physically dictate where the agent moves next. It mathematically determines *how* the entity traverses the HEP surface before calling the lower-level **update_pos** infrastructure to register the move.
- **agent_above_water**: A scientific validation function that checks ecological boundaries (is the agent's coordinate currently designated as water?). If so, it returns true, which allows higher science functions to dictate a response (like death or bouncing back).

By structuring the code across these three tiers, researchers can freely modify the high-level scientific equations without risking memory leaks or index corruption.

2.6 Clustering and Spatial Analysis

The simulation supports several algorithms for identifying agent populations and density peaks.

2.6.1 K-Means Clustering

Core K-Means clustering is implemented in `mod_kmeans.f95`. **Note on Initialization**: To ensure uniform coverage and isolation of geographically remote populations, the framework utilizes a **Farthest-First Traversal (K-means++)** algorithm for centroid initialization. This prevents redundant clustering within a single high-density peak when multiple remote agent islands exist.

2.6.2 DBSCAN (Density-Based Spatial Clustering)

DBSCAN is available for discovering clusters of arbitrary shapes and identifying noise (outliers).

- **Epsilon (ϵ)**: The maximum distance between two samples for one to be considered as in the neighborhood of the other.
- **MinPts**: The number of samples in a neighborhood for a point to be considered as a core point.

2.6.3 Convex Hull Fill

To improve visual consistency, a **Convex Hull Post-Processing** step is applied to clustered grid cells. This fills internal holes and noise gaps within identified clusters, preventing cross-cluster contamination while maintaining structural integrity.

2.7 Visualization and Rendering

The Python visualization engine (`simulation.py`) handles the translation of Fortran grid data into interactive maps.

2.7.1 Rendering Architecture

- **Bifurcated Loops:** 2D and 3D visual rendering are structurally independent. The `on_sim_progress` method utilizes different mathematical paths for drawing flat grids (`ImageItem`) versus interpolating spherical terrains (`GLMeshItem`).
- **RGBA Matrices:** The backend simulation pipeline expects arrays formatted exactly as **4-channel RGBA** (R, G, B, Alpha). Boundary edges and UI markers are "burned" directly into these matrices inside Python to ensure rendering certainty.

2.8 Best Practices

- **Statelessness:** Keep modules stateless where possible. Store state in agent fields or in module-level `save` variables.
- **Grid Access:** Use `w%grid` to access spatial neighbour information.
- **Performance:** Avoid file I/O inside per-agent loops. Use `print` sparingly (e.g., only on specific ticks).
- **Debug Output:** Always call `flush(6)` after `print` statements to ensure output appears before a potential crash.

Chapter 3

Data Structures & Storage

3.1 Data Storage Logic

Efficiently storing and retrieving agents is critical for performance. The framework uses a **Structure-of-Arrays (SoA)** approach mixed with object-oriented containers to balance cache efficiency and code maintainability.

3.1.1 Design Choice: Hybrid Storage

The simulation needs to handle millions of agents. Storing each agent as a separate object scattered in memory (as in a pure linked list) would cause excessive cache misses. Conversely, a pure structure-of-arrays can make code harder to read. We compromise by using a `world_container` that holds global arrays, but we expose individual agents via pointers to derived `Agent` types that "view" the array data.

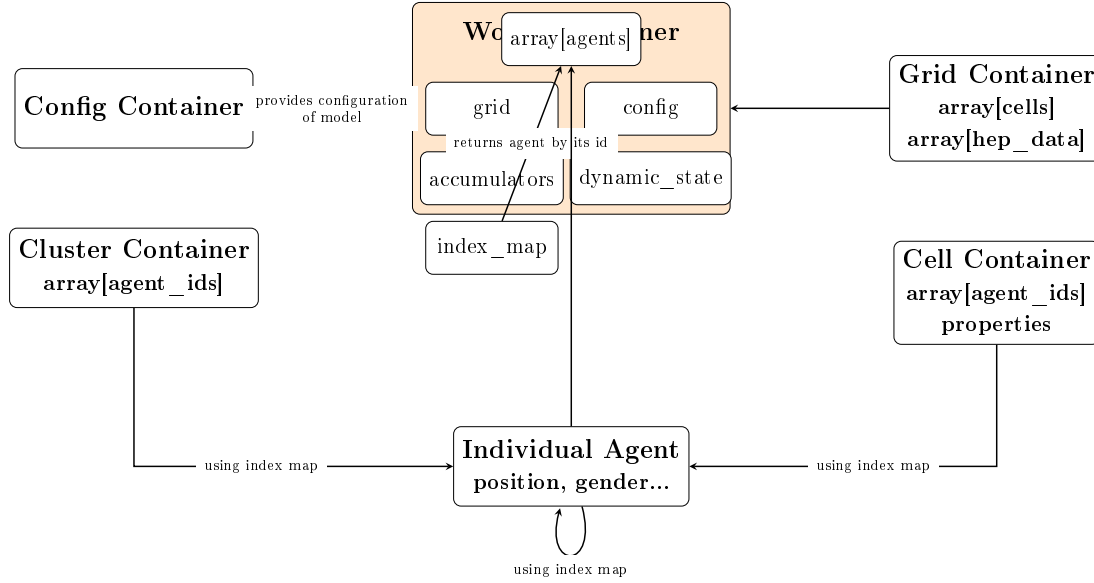


Figure 3.1: Visual representation of the hybrid data structure. The `world_container` acts as the root, managing contiguous memory arrays for agents while maintaining spatial (Grid) and logical (Hashmap) indices for efficient access.

3.1.2 The Agent Array

Agents are stored in a large, pre-allocated 2D array within the `world_container`:

```
1 type(Agent), allocatable :: agents(:, :) ! (max_size, npops)
```

- **Rows:** Iterate through individual agents (up to `max_size`).
- **Columns:** Separate different populations (e.g., `npops`).

This allows for contiguous memory access when iterating over a single population, which is the most common operation.

3.1.3 Spatial Indexing: The Grid

For spatial interactions (e.g., mating, conflict), agents are indexed by their geospatial location using the **Grid** structure.

- Each grid cell (`grid_cell`) contains a dynamic list of Agent IDs currently located in that cell.
- When an agent moves, it is removed from the old cell's list and added to the new cell's list.

Module Types: Agent-Centric vs Cell-Centric

When developing modules, you must decide the primary access pattern:

- **Agent-Centric Modules:** Functions that apply to a specific agent (e.g., Movement). These iterate over the `agents` array.

```
1     subroutine agent_move(agent_ptr)
2         type(Agent), pointer :: agent_ptr
3     
```

- **Cell-Centric Modules:** Functions that apply to a location (e.g., Overpopulation Death). These iterate over the `grid`.

```
1     subroutine death_overpopulation(cell_ptr)
2         type(cell), pointer :: cell_ptr
3     
```

3.2 Global State & Metrics

In addition to individual agent data, the `world_container` tracks macroscopic simulation states and per-tick accumulators to monitor population health and system performance.

3.2.1 Tick Accumulators (`t_tick_accumulators`)

A history of accumulators is maintained (`accumulators_history(:)`) to track events within a single tick. These are automatically reset at the start of each step:

- `phi_birth_acc` / `phi_death_acc`: Cumulative birth and death counts.
- `n_alive_acc`: Count of agents currently alive.
- `ms_per_tick`: Execution time for the current simulation step.

3.2.2 Dynamic State (`t_dynamic_state`)

Global variables that evolve over time and influence agent behavior:

- `K_fertility`: A macroscopic scaling factor used to regulate population growth according to the Verhulst model.

3.3 Module `mod_agent_world`

This module defines the central `world_container` which holds:

- `agents`: The main agent array.
- `grid`: The spatial grid.
- `index_map`: The ID lookup hashmap.
- `config`: Global configuration parameters.
- `accumulators_history`: Buffer of recent performance and biological metrics.
- `dynamic_state_vars`: Current macroscopic simulation state.

It serves as the "Hub" for all data access.

3.3.1 Agent Lookup: Hashmap

To find an agent by its unique ID (without searching the entire array), the system uses a custom integer hashmap (`mod_hashmap`).

- **Key**: Agent ID (Integer).
- **Value**: A composed integer packing both `population_index` and `array_index`.

This allows $O(1)$ access to any agent pointer given its ID.

3.4 Compaction

When agents die, they are marked with `is_dead = .true..` They are NOT removed immediately to avoid array resizing costs. Instead, a `compact_agents` subroutine is called periodically (e.g., end of timestep).

1. It iterates from both ends of the array.
2. Dead agents on the left are overwritten by living agents from the right.
3. The array size is logically reduced (tracking integer), but physically remains allocated.

3.5 Density Analysis & Clustering

The simulation includes a suite of spatial analysis modules (`mod_watershed`, `mod_kmeans`, `mod_clustering`) that group grid cells into clusters based on agent density and HEP suitability.

3.5.1 Global Density Smoothing

To ensure consistency across different clustering algorithms, a unified density surface (`human_density_smoothed`) is generated once per tick. This surface is processed using an iterative box filter.

3.5.2 Algorithms

1. **Watershed**: Performs gradient ascent on the global density surface.
2. **K-Means++**: Utilizes a **Farthest-First Traversal** initialization to ensure centroids are placed in remote agent populations, avoiding redundant clustering on a single peak.
3. **Convex Hull Fill**: A post-processing step that fills internal holes in clusters, ensuring structural integrity for visualization and demographic analysis.

3.5.3 Configuration

Clustering parameters are defined in the `namelist /config/` block:

Parameter	Type	Default	Description
<code>human_density_smoothing_radius</code>	integer	2	Radius of the box-filter applied to the global density surface.
<code>human_density_smoothing_iterations</code>	integer	1	Number of iterative passes for the box-filter (high-pass smoother).
<code>watershed_threshold</code>	real(8)	0.05	Minimum density for a cell to be included in a cluster.

Chapter 4

Fortran Only Version

The simulation framework includes a standalone Fortran executable, allowing users to run the model without the Python interface or GUI. This is particularly useful for high-performance computing (HPC) environments, batch processing, or profiling.

4.1 Building the Executable

The standalone version is built using the provided shell script `build_fortran.sh`. This script compiles all necessary modules, creates a static library (`libhep.a`), and links the final executable.

4.1.1 Prerequisites

- **Compiler:** `gfortran` (or compatible Modern Fortran compiler).
- **Libraries:** NetCDF and NetCDF-Fortran development libraries (`libnetcdf-dev`, `libnetcdf-f-dev`).

4.1.2 Build Command

To build the project, run the following command from the project root:

```
1 ./build_fortran.sh
```

This will create:

1. A `build/` directory containing compiled object files (`.o`) and module files (`.mod`).
2. A static library `build/libhep.a`.
3. The standalone executable `build/main_fortran`.

4.2 Running the Simulation

The executable can be run directly from the command line. It reads the specific configuration file located at `input/config/main_fortran_config.nml`.

4.2.1 Execution

```
1 ./build/main_fortran
```

4.2.2 Command Line Arguments

The standalone version supports command-line arguments to override specific runtime parameters:

- `-output_interval <integer>`: Sets the interval (in ticks) at which simulation status is printed to the console. Default is 1000.

Example:

```
1 ./build/main_fortran --output_interval 500
```

4.3 Configuration

Unlike the Python version which uses `basic_config.nml`, the standalone Fortran version hardcodes the configuration path to:

`input/config/main_fortran_config.nml`

Ensure this file exists and contains valid namelists (`&dims` and `&config`) before running. The structure of this file is identical to the standard configuration files used by the GUI.

Chapter 5

Recent Changes

Chapter 6

Project State Summary - May 5, 2026

6.1 Objective

The primary goals addressed recently were the implementation of **Auto K-Means and DBSCAN**, resolving simulation stability issues, performing a general codebase cleanup, and standardizing the **Birth-Death simulation modules** for consistent scientific modeling.

6.2 Completed Work

6.2.1 1. Dead Agent Archive & NetCDF Storage (April 21st)

- **Objective:** Implement a performance-conscious system for archiving deceased agents for historical analysis.
- **Work Done:**
 - **Fortran Backend:** Added `store_dead_agents` flag to `world_container` and logic to archive agent data before deletion.
 - **Asynchronous Pipeline:** Built an async Python writer using `multiprocessing.Queue` to persist data to NetCDF without blocking the simulation thread.
 - **UI Integration:** Added a checkbox in the "Full Simulation" tab and status indicators for the writer queue.
 - **Bug Fixes:** Resolved segfaults in the F2PY interface by refactoring agent extraction into a clean dataclass-based pipeline.

6.2.2 2. New Module Pattern & World Data Access (April 23rd)

- **Objective:** Standardize how new agent-logic modules are added and how they interact with global state.
- **Work Done:**
 - **Accumulator Reset:** Established a mechanism for `t_tick_accumulators` that automatically reset at the end of each simulation tick.
 - **Module Registration:** Defined a clear 3-step pattern for adding modules (Fortran ID -> Python Interface -> SpawnPointEditor registration) following the `reviewed_agent_motion` template.
 - **World Access:** Identified and documented accessible agent variables via the `world` object to facilitate custom agent-logic development.

6.2.3 3. DBSCAN, Convex Hull & Performance Metrics (April 28th)

- **Objective:** Expand clustering options, improve cluster integrity, and track simulation efficiency.
- **Work Done:**
 - **DBSCAN Implementation:** Added point-based and grid-based DBSCAN clustering in Fortran.
 - **UI Controls:** Added dynamic control over DBSCAN parameters (`eps`, `minPts`) directly from the Python GUI.
 - **Convex Hull Fill:** Integrated a convex hull post-processing step into the grid-based voting mechanism to eliminate holes and noise gaps within identified clusters.
 - **Performance Monitoring:** Implemented `avg_ms_per_tick` tracking. The average time per tick is now displayed in the window title and is available as a plottable variable in the time-series UI.

6.2.4 4. Unified Density Smoothing & Global Metrics (April 29th)

- **Objective:** Standardize density smoothing across all clustering modes and enable real-time plotting of global states.

- **Work Done:**

- **Unified Smoothing:** Replaced redundant local smoothing in Auto K-Means with a global `human_density_smoothing_radius` parameter.
- **Iterative Smoothing:** Added `human_density_smoothing_iterations` to allow for aggressive, multi-pass global smoothing.
- **Global Metrics:** Exported `t_dynamic_state` and `t_tick_accumulators` to Python. Added variables like `k_fertility (sim)`, `phi_death_acc (sim)`, and various `death_* (sim)` debug counters to the UI plotting engine.
- **Dual-Axis Fix:** Resolved a PyQtGraph closure bug where secondary axes caused the plot canvas to collapse.

6.2.5 5. Birth-Death Refactoring & Logic Consolidation (May 4th)

- **Objective:** Standardize parameter naming and consolidate biological logic for easier scientific review.
- **Work Done:**
 - **Parameter Schema Migration:** Replaced legacy `b1-b4` identifiers with standardized ecological parameters: `r` (growth rate), `NC` (carrying capacity), `Kmin`, and `Kmax`.
 - **Module Consolidation:** Migrated birth/death logic from the obsolete `mod_birth_death_new.f95` into the reviewed and consolidated `mod_reviewed_modules.f95`.
 - **Build System Sync:** Updated the dependency graph and Python interface to support the new parameter schema and modular structure.

6.2.6 6. Architectural Refinement & Critical Fixes (April - May)

- **K-Means Farthest-First Traversal:** Replaced the peak-value search in K-Means with a **Farthest First Traversal (K-means++)** subroutine to ensure geographically remote populations are isolated uniformly.

- **Cluster ID 1-Indexing:** Fixed a visualization bug where Cluster ID 0 was being masked as noise; the system now uses 1-based indexing for all valid clusters.
- **Rendering & Matrix Edge Geometry:** Resolved line rendering issues by translating cluster maps directly through NumPy boolean slicing filters and "burning" 1-pixel white boundaries into the RGBA output matrices.
- **3D Visualization:** Replaced unstable OpenGL 3D views with Matplotlib 3D surfaces in the `compare_smoothing.py` tool for robust cluster verification in headless environments.
- **F2PY Namespace Hotfix:** Fixed a critical bug where localized imports of `mod_python_interface` shadowed the extension module, causing `AttributeError` crashes in the update loop.

6.3 TODO later

- **[LATEX/DOCS]:** Add an entry explaining the Farthest-First Traversal algorithm.
- **[PYTHON/DOCS]:** Document that visual rendering loops for 2D and 3D are fundamentally separate.
- **[PYTHON/DOCS]:** Mark that the backend expects exactly 4-channel RGBA arrays for map updates.

6.3.1 7. Cluster Population Control & Multi-Population Graphing (May 13th)

- **Objective:** Resolve population convergence discrepancies between global and cluster-based modules and improve the Python graphing capabilities.
- **Work Done:**
 - **Divergence Analysis:** Identified that the cluster-based module (`new_birth`) was severely under-suppressing births compared to the global (`reviewed_birth`) module. This was caused by the cluster module relying on the per-cluster accumulator `n_alive_acc(jp)`, which structurally misses agents not assigned to any cluster (e.g. `c_idx = 0`), resulting in massive population under-estimation.

- **Debug Tooling:** Implemented a global-vs-cluster diagnostic summation script within `python_interface.f95` to track the exact magnitude of unassigned agents escaping the control logic.
- **Dynamic UI Plotting via Population Filters:**
- Enhanced `mod_python_interface.get_dynamic_state_stats` and `get_cluster_info` to optionally accept and return stats based on a specific `jp` population index.
- Added a dedicated `pop:` spinbox to the Python GUI (in `application.py`) that feeds into the graph configuration.
- Implemented **"Dominant Population"** (`pop = -1`) logic, allowing graphs to dynamically query the backend tick-by-tick and automatically select and plot statistics (`NC`, `n_agents`, `k_fertility`) specifically for whichever population currently has the most agents within the targeted cluster or globally.

6.3.2 8. Performance Metrics HUD & Population-Specific Mating (May 26th)

- **Objective:** Add real-time wall-clock profiling to identify simulation bottlenecks, and introduce optional reproductive isolation between populations.
- **Work Done:**
 - **Fortran Instrumentation:** Wrapped each phase of `step_simulation` in `python_interface.f95` with `system_clock` calls to measure wall-clock time for permanent modules, active modules, compaction, grid/density, and clustering.
 - **API:** Exposed `set_performance_timing_enabled` and `get_performance_s` to Python via `mod_python_interface`. Implemented as optional with **zero overhead** when disabled.
 - **GUI HUD:** Added a “Show Performance Metrics” checkbox in the “View Editor” tab and a glassmorphic tech-cyan overlay in the simulation window displaying per-phase timing breakdowns.
 - **Population-Specific Mating** (`allow_across_populations`): Added a new config flag `allow_across_populations` (default `.true.`). When set to `.false.`, the `get_male_from_cell` function is called with `only_population=jp`, enforcing strict reproductive isolation so that females can only mate with males of the same population.

- **Files Modified:** `python_interface.f95`, `mod_agent_world.f95`, `mod_reviewed_modules.f95`, `mod_birth_death_new.f95`, `mod_config.f95`, `mod_read_inputs.f95`, `basic_config.nml`, `config_sandesh.nml`, `simulation.application.py`.

6.3.3 9. Critical Bugfix: Hashmap Population Corruption (May 26th)

- **Objective:** Fix a bug causing phantom agents of incorrect populations (e.g. Pop 1 red dots) to appear in single-population simulations.
- **Root Cause:** The `remove()` subroutine in `mod_hashmap.f95` used **open-address linear probing**. When an entry was deleted, subsequent entries in the probe chain were rehashed to fill the hole. The rehash code correctly saved and restored the **key** and **value** (array index) fields, but **failed to save or restore the population field**. The moved entry inherited a stale population value from the vacated bucket.
- **Impact:** When `get_agent(id, world)` later looked up a rehashed agent via `get_index_and_pop`, it received an incorrect population index. This caused an **out-of-bounds array access** into `agents(index, stale_population)`, reading garbage memory that could appear as an agent with `population = 1`, rendered as a red dot in the visualization.
- **Fix Applied** (4 changes to `mod_hashmap.f95`):
 - Added `temp_population` variable to the rehash loop.
 - Clear `population = -1` when initially removing an entry's bucket.
 - Save `temp_population` alongside `temp_key/temp_value` when reading entries to move.
 - Write `temp_population` to the destination bucket and clear `population = -1` on the source bucket after moving.
- **File Modified:** `src/data_structures/mod_hashmap.f95`.

6.3.4 10. Efficient Cluster-Restricted Density Updates (May 26th)

- **Objective:** Speed up the main bottleneck of the simulation step (the $O(N)$ grid sweeps for raw density, combined agent flows, box smooth-

ing, and carrying capacity copies) by restricting updates to cells that belong to active clusters.

- **Work Done:**

- **New Subroutine `update_density_and_hep_grid_efficient`:** Implemented a cluster-restricted variant of the full density update in `mod_technical_modules.f95`. Instead of sweeping all $n_x \times n_y$ cells, it iterates only over cells belonging to active clusters (`cluster_store%clusters(k)%cell_gx/gy`), computing raw density, agent flows, and HEP copies exclusively for those cells.
- **Hybrid Scheduling in `step_simulation`:** Modified the density update dispatch in `python_interface.f95` to choose between the full and efficient variants based on tick alignment. On clustering ticks (`mod(t, update_interval) == 0`), the legacy `update_density_and_hep_grid` runs to ensure the full grid is populated before re-clustering. On intermediate ticks, the efficient variant is used.
- **Boundary Orphan Reset:** Designed and implemented a `was_clustered(:)` boundary logical grid array inside the `Grid` type. On re-clustering ticks (`mod(t, update_interval) == 0`), any cell that transitioned OUT of a cluster is immediately zeroed out.
- **$O(1)$ Neighborhood box smoothing:** Restricted the smoothed density box-filter calculations to cluster cells, looking up neighbor cells directly (which naturally evaluate to 0.0 outside clusters).
- **$O(1)$ HEP data copies:** Restricted carrying capacity copies strictly to cells within active clusters.
- **Conditional Startup Fallback:** Embedded conditional fallback inside `step_simulation` in `python_interface.f95` to automatically pivot to the legacy `update_density_and_hep_grid` on tick 0 or when no clusters exist, ensuring robust carrying capacity (NC) initialization and preventing agent die-offs.
- **Config Flag:** Gated behind `efficient_density_updates` (boolean) in the namelist. Currently set to `.false.` in both `basic_config.nml` and `config_sandesh.nml` pending validation that it does not affect simulation accuracy.

- **Status:** Implemented and compiles, but **currently disabled** (`efficient_density_updates = .false.`).

- **Files Modified:** `python_interface.f95`, `mod_technical_modules.f95`, `mod_grid_id.f95`, `mod_config.f95`, `mod_read_inputs.f95`, `basic_config.nml`, `config_sandesh.nml`.

6.3.5 11. Responsive Rolling Average for Performance HUD (May 26th)

- **Objective:** Upgrade the performance metrics HUD from a global cumulative average to a responsive rolling average that forgets the distant past and immediately reacts to mid-run toggle switches.
- **Work Done:**
 - **20-Tick Sliding Window Buffer:** Designed and implemented a module-level sliding circular buffer array `perf_history(6, 20)` in `python_interface.f95` that records the exact timings of the last 20 ticks.
 - **Reset on Toggle:** Programmed the rolling timing history array to completely clear and restart fresh whenever performance timing is enabled or disabled in `set_performance_timing_enabled`.
 - **Arithmetic Average stats:** Updated `get_performance_stats` to calculate the exact, unweighted arithmetic average of the recorded window (up to 20 samples), returning clean rolling averages without any clearing side-effects.
- **Files Modified:** `src/interfaces/python_interface.f95`.

6.3.6 12. HEP Data Chunked Loading (May 26th)

- **Objective:** Reduce memory footprint for large HEP NetCDF files (≈ 900 MB) by loading only a window of time slices instead of the entire dataset.
- **Work Done:**
 - **Dynamic Chunk Sizing:** `read_hep_data` in `mod_read_inputs.f95` now queries the HEP file size and calculates a `slices_per_chunk` count targeting ≈ 10 MB chunks. Only the first chunk is loaded at startup.
 - **On-Demand Re-Loading:** Added `load_hep_chunk_from_file` subroutine. When `t_hep` advances past the current chunk's range

(`chunk_start_t..chunk_end_t`), a new chunk is loaded from the NetCDF file with the correct `start` offset.

- **Local Index Translation:** `update_density_and_hep_grid` translates the global `t_hep` to a local chunk index via `local_idx = t_hep - chunk_start_t + 1` before reading `grid%hep(:, :, jp, local_idx)`.
- **Grid Type Extension:** Added `slices_per_chunk`, `chunk_start_t`, `chunk_end_t` metadata fields to the `Grid` type in `mod_grid_id.f95`, and a `config` pointer so the chunk loader can access file paths.
- **Files Modified:** `mod_read_inputs.f95`, `mod_grid_id.f95`, `mod_agent_world.f95`, `mod_technical_modules.f95`.

6.3.7 13. Clustering Fix: “Clusters Way Too Small” (May 26th)

- **Objective:** Diagnose and fix the issue where watershed clusters were spatially too tight, capturing only $\approx 55\%$ of alive agents and missing $\approx 45\%$ in peripheral cells.
- **Root Cause:** Three interacting factors:
 1. `watershed_threshold` was changed from `0.00` $\rightarrow$ `0.05`: With the old value, every non-zero-density cell was included. The new value excluded peripheral cells where smoothed density fell below `0.05`.
 2. `human_density_smoothing_radius` was changed from `3` $\rightarrow$ `5`: The wider kernel spreads density further, reducing peak values and causing more edge cells to drop below threshold.
 3. Redundant internal smoothing in `watershed_cluster()`: The watershed algorithm was applying 4 additional iterations of box-filter smoothing (`radius=4`) on top of the already-smoothed `human_density_surface`.
- **Fix Applied:**
 - Lowered `watershed_threshold` to `0.01` in both `basic_config.nml` and `config_sandesh.nml`.
 - Removed the redundant 4-iteration internal smoothing loop from `watershed_cluster()` in `mod_watershed.f95`.
- **Files Modified:** `mod_watershed.f95`, `basic_config.nml`, `config_sandesh.nml`

6.3.8 14. Bugfix: Out-of-Bounds Array Access in Cluster Accumulators (May 26th)

- **Objective:** Fix memory corruption caused by accessing `cell_cluster_idx` with invalid grid coordinates.
- **Root Cause:** Agents have a default `gx = -1` before being placed on the grid. The accumulator code in `update_agent_age` (`mod_technical_modules.f95`) accessed `cell_cluster_idx(agent%gx, agent%gy)` without bounds-checking. With `gx = -1`, this is an **out-of-bounds Fortran array access** that reads arbitrary memory.
- **Same bug in debug code:** The detailed debug breakup prints in `python_interface.f95` had the identical issue.
- **Fix Applied:**
 - Added `gx >= 1 .and. gx <= nx .and. gy >= 1 .and. gy <= ny` bounds check in `update_agent_age` before accessing `cell_cluster_idx`.
 - Added `c_idx <= n_clusters` upper-bound check before using `c_idx` as an index into `clusters(:)`.
 - Added matching bounds checks in the debug breakup code in `python_interface.f95`.
- **Files Modified:** `mod_technical_modules.f95`, `python_interface.f95`.

6.4 Tripwires & Lessons Learned

- **1. Fortran Array Out-of-Bounds is Silent:** Fortran does **not** bounds-check array accesses by default. Accessing `array(-1, 5)` doesn't crash — it silently reads garbage from adjacent memory. This garbage then propagates through the simulation as valid data.
- **2. Config Changes Can Have Non-Obvious Algorithmic Interactions:** The `watershed_threshold` increase from 0.00 to 0.05 seemed harmless — but it interacted catastrophically with the independently-added 4x internal smoothing in the watershed algorithm.
- **3. “Parity” Code Can Be Counter-Productive:** The internal smoothing loop was added to bring watershed clustering into “parity” with Auto K-Means. But the two algorithms have fundamentally different sensitivities.

- **4. Misleading Debug Output Can Send You in Circles:** The debug diagnostic showing “316 agents with invalid/unallocated indices” looked like a clustering algorithm failure. In reality, it was the debug code itself having an out-of-bounds bug.
- **5. `efficient_density_updates = .false.` Made Fix Attempts Into No-Ops:** The earlier fix attempt (hybrid density scheduling, `was_clustered` grid cleanup) was architecturally sound but was entirely guarded by `efficient_density_updates`, which was `.false.` in both configs.